

LEX

```
%{  
  
#include <stdio.h>  
  
#include <stdlib.h>  
  
#include <string.h>  
  
#include "lang.tab.h"  
  
  
%}  
  
  
%option noyywrap  
%option yylineno  
  
  
%x COMMENT  
  
  
DIGIT    [0-9]  
LETTER   [a-zA-Z]  
ID_FIRST [a-zA-Z]  
ID_FOLLOW [a-zA-Z0-9_]  
  
  
%%  
  
      /* ----- Reserved Words ----- */  
"array" { printf("TOKEN: ARRAY\n"); return ARRAY; }
```

```
"char"    { printf("TOKEN: CHAR\n"); return CHAR; }
"const"   { printf("TOKEN: CONST\n"); return CONST; }
"else"    { printf("TOKEN: ELSE\n"); return ELSE; }
"if"      { printf("TOKEN: IF\n"); return IF; }
"int"     { printf("TOKEN: INT\n"); return INT; }
"of"      { printf("TOKEN: OF\n"); return OF; }
"read"    { printf("TOKEN: READ\n"); return READ; }
"while"   { printf("TOKEN: WHILE\n"); return WHILE; }
"print"   { printf("TOKEN: PRINT\n"); return PRINT; }
"string"  { printf("TOKEN: STRING\n"); return STRING; }
```

```
/* ----- Operators & Delimiters ----- */
```

```
"=="      { printf("TOKEN: EQ\n"); return EQ; }
"!="      { printf("TOKEN: NEQ\n"); return NEQ; }
"<="      { printf("TOKEN: LE\n"); return LE; }
">="      { printf("TOKEN: GE\n"); return GE; }
"<"       { printf("TOKEN: LT\n"); return LT; }
">"       { printf("TOKEN: GT\n"); return GT; }
"="       { printf("TOKEN: ASSIGN\n"); return ASSIGN; }
"+"       { printf("TOKEN: PLUS\n"); return PLUS; }
"-"       { printf("TOKEN: MINUS\n"); return MINUS; }
"*"       { printf("TOKEN: MULT\n"); return MULT; }
"/"       { printf("TOKEN: DIV\n"); return DIV; }
"%"       { printf("TOKEN: MOD\n"); return MOD; }
"("       { printf("TOKEN: LPAREN\n"); return LPAREN; }
")"       { printf("TOKEN: RPAREN\n"); return RPAREN; }
```

```
"{" { printf("TOKEN: LBRACE\n"); return LBRACE; }
"}" { printf("TOKEN: RBRACE\n"); return RBRACE; }
"[" { printf("TOKEN: LBRACK\n"); return LBRACK; }
"]" { printf("TOKEN: RBRACK\n"); return RBRACK; }
";" { printf("TOKEN: SEMI\n"); return SEMI; }
```

```
/* ----- Comments ----- */
```

```
#[^\n]* { /* ignore comments starting with '#' */ }
```

```
/* ----- Identifiers ----- */
```

```
{ID_FIRST}{ID_FOLLOW}* {
    yylval.sval = strdup(yytext);
    printf("TOKEN: IDENTIFIER (%s)\n", yytext);
    return IDENTIFIER;
}
```

```
/* ----- Integer Constants ----- */
```

```
{DIGIT}+ {
    yylval.ival = atoi(yytext);
    printf("TOKEN: INT_CONST (%s)\n", yytext);
    return INT_CONST;
}
```

```
/* ----- Character Constants ----- */
```

```
'[a-zA-Z0-9_]'\ {
    yylval.ival = yytext[1];
```

```

printf("TOKEN: CHAR_CONST (%s)\n", yytext);
return CHAR_CONST;
}

```

```

/* ----- String Constants ----- */
\[^\"]*\" {
    size_t len = strlen(yytext);
    char *str = (char *)malloc(len);
    strncpy(str, yytext + 1, len - 2);
    str[len - 2] = '\0';
    yylval.sval = str;
    printf("TOKEN: STRING_CONST (%s)\n", str);
    return STRING_CONST;
}

```

```

/* ----- Whitespace ----- */
[\t]+ { /* Skip spaces and tabs */ }
\n { yylineno++; }

```

```

/* ----- Unknown Characters ----- */
. { fprintf(stderr, "Unexpected character '%s' on line %d\n", yytext, yylineno); }

```

```

%%

```

YACC:

```
%{  
  
#include <stdio.h>  
  
#include <stdlib.h>  
  
#include <string.h>  
  
#include "lang.tab.h"  
  
  
extern FILE *yyin;  
  
extern int yylineno;  
  
  
void yyerror(const char *s);  
  
int yylex(void);  
  
  
%}  
  
  
%union {  
  
    int ival; // For integer constants or character codes  
  
    char *sval; // For string constants or identifiers  
  
}  
  
  
%token <sval> IDENTIFIER STRING_CONST  
  
%token <ival> INT_CONST CHAR_CONST  
  
%token ARRAY CHAR CONST ELSE IF INT OF READ WHILE PRINT STRING  
  
%token PLUS MINUS MULT DIV MOD ASSIGN LT LE EQ NEQ GE GT
```

%token LPAREN RPAREN LBRACE RBRACE LBRACK RBRACK SEMI COMMENT

%type <sval> print_type

%type <ival> expression factor

%%

program

: LBRACE statement_list RBRACE

```
{  
    printf("\n Parsing completed successfully.\n");  
}
```

;

statement_list

: statement_list statement

```
{ printf("PRODUCTION: statement_list -> statement_list statement\n"); }
```

| statement

```
{ printf("PRODUCTION: statement_list -> statement\n"); }
```

;

statement

: simple_statement SEMI

```
{ printf("PRODUCTION: statement -> simple_statement ;\n"); }
```

| struct_statement

```
{ printf("PRODUCTION: statement -> struct_statement\n"); }
```

| comment

```
{ printf("PRODUCTION: statement -> comment ;\n"); }  
;
```

comment: COMMENT

```
;
```

/* ----- Simple Statements ----- */

simple_statement

: read_statement

```
{ printf("PRODUCTION: simple_statement -> read_statement\n"); }
```

| print_statement

```
{ printf("PRODUCTION: simple_statement -> print_statement\n"); }
```

| assign_statement

```
{ printf("PRODUCTION: simple_statement -> assign_statement\n"); }
```

| declaration_statement

```
{ printf("PRODUCTION: simple_statement -> declaration_statement\n"); }
```

```
;
```

read_statement

: READ LPAREN IDENTIFIER RPAREN

```
{
```

```
    printf("PRODUCTION: read_statement -> read ( identifier )\n");
```

```
}
```

```
;
```

print_statement

: PRINT LPAREN print_type RPAREN

```
{  
    printf("PRODUCTION: print_statement -> print ( print_type )\n");  
}  
;
```

print_type: IDENTIFIER { \$\$ = \$1; }

| STRING_CONST { \$\$ = \$1; }

| INT_CONST { \$\$ = strdup("integer"); }

| CHAR_CONST { \$\$ = strdup("character"); }

;

assign_statement: IDENTIFIER ASSIGN expression

```
{  
    printf("PRODUCTION: assign_statement -> identifier = expression\n");  
}  
;
```

declaration_statement

: type IDENTIFIER

```
{  
    printf("PRODUCTION: declaration_statement -> type identifier\n");  
}  
;
```

```
/* ----- Types ----- */
```

```
type
```

```
  : simple_type
```

```
    { printf("PRODUCTION: type -> simple_type\n"); }
```

```
  | arraydecl
```

```
    { printf("PRODUCTION: type -> arraydecl\n"); }
```

```
;
```

```
simple_type
```

```
  : STRING
```

```
    { printf("simple_type -> string\n"); }
```

```
  | INT
```

```
    { printf("simple_type -> int\n"); }
```

```
  | CHAR
```

```
    { printf("simple_type -> char\n"); }
```

```
;
```

```
arraydecl
```

```
  : ARRAY LBRACK INT_CONST RBRACK simple_type
```

```
    {
```

```
      printf("arraydecl -> array [ no ] simple_type\n");
```

```
    }
```

```
;
```

```
/* ----- Structural Statements ----- */
```

```
struct_statement
```

```
: decision_statement
```

```
{  
    printf("struct_statement -> decision_statement\n");  
}
```

```
| while_statement
```

```
{  
    printf("struct_statement -> while_statement\n");  
}
```

```
;
```

```
decision_statement
```

```
: if_statement else_clause
```

```
{  
    printf("decision_statement -> if_statement else_clause\n");  
}
```

```
;
```

```
if_statement
```

```
: IF LPAREN condition RPAREN LBRACE statement_list RBRACE
```

```
{  
    printf("if_statement -> if ( condition ) { statement_list }\n");  
}
```

```
;
```

else_clause

: ELSE LBRACE statement_list RBRACE

```
{  
    printf("else_clause -> else { statement_list }\n");  
}
```

| /* epsilon */

```
{  
    printf("else_clause -> epsilon\n");  
}
```

;

while_statement

: WHILE LPAREN condition RPAREN LBRACE statement_list RBRACE

```
{  
    printf("while_statement -> while ( condition ) { statement_list }\n");  
}
```

;

/* ----- Condition ----- */

condition

: expression relation expression

```
{  
    printf("condition -> expression relation expression\n");  
}
```

;

relation

```
: LT { printf("relation -> <\n"); }
| LE { printf("relation -> <=\n"); }
| EQ { printf("relation -> ==\n"); }
| NEQ { printf("relation -> !=\n"); }
| GE { printf("relation -> >=\n"); }
| GT { printf("relation -> >\n"); }
;
```

```
/* ----- Expressions ----- */
```

```
expression: factor int_expression_rest
;
```

```
int_expression_rest: PLUS expression
```

```
    | MINUS expression
```

```
    | MULT expression
```

```
    | DIV expression
```

```
    | MOD expression
```

```
    | /* epsilon */
```

```
;
```

```
factor: INT_CONST    { $$ = $1; }
```

```
    | IDENTIFIER    { $$ = 0; } // Placeholder for identifiers
```

```
    | CHAR_CONST    { $$ = $1; }
```

```
    | STRING_CONST  { $$ = 0; } // Placeholder for string literals
```

```
;
```

%%

```
void yyerror(const char *s) {  
    fprintf(stderr, "Syntax error on line %d: %s\n", yylineno, s);  
}
```

```
int main(int argc, char **argv) {  
    if (argc > 1) {  
        FILE *f = fopen(argv[1], "r");  
        if (!f) {  
            perror(argv[1]);  
            return 1;  
        }  
        yyin = f;  
    }  
    yyparse();  
    return 0;  
}
```

DEMO:

p1.txt:

```
{  
# aaaS;
```

```
int a;
```

```
int b;
```

```
int c;
```

```
int max;
```

```
read(a);
```

```
read(b);
```

```
read(c);
```

```
if (a >= b) {
```

```
    if (a >= c) {
```

```
        max = a;
```

```
    } else {
```

```
        max = c;
```

```
    }
```

```
} else {
```

```
    if (b >= c) {
```

```
        max = b;
```

```
    } else {
```

```
        max = c;
```

```
    }
```

```
}
```

```
print(max);
```

```
}
```

OUTPUT:

TOKEN: LBRACE

TOKEN: INT

simple_type -> int

PRODUCTION: type -> simple_type

TOKEN: IDENTIFIER (a)

PRODUCTION: declaration_statement -> type identifier

PRODUCTION: simple_statement -> declaration_statement

TOKEN: SEMI

PRODUCTION: statement -> simple_statement ;

PRODUCTION: statement_list -> statement

TOKEN: INT

simple_type -> int

PRODUCTION: type -> simple_type

TOKEN: IDENTIFIER (b)

PRODUCTION: declaration_statement -> type identifier

PRODUCTION: simple_statement -> declaration_statement

TOKEN: SEMI

PRODUCTION: statement -> simple_statement ;

PRODUCTION: statement_list -> statement_list statement

TOKEN: INT

simple_type -> int

PRODUCTION: type -> simple_type

TOKEN: IDENTIFIER (c)

PRODUCTION: declaration_statement -> type identifier

PRODUCTION: simple_statement -> declaration_statement

TOKEN: SEMI

PRODUCTION: statement -> simple_statement ;

PRODUCTION: statement_list -> statement_list statement

TOKEN: INT

simple_type -> int

PRODUCTION: type -> simple_type

TOKEN: IDENTIFIER (max)

PRODUCTION: declaration_statement -> type identifier

PRODUCTION: simple_statement -> declaration_statement

TOKEN: SEMI

PRODUCTION: statement -> simple_statement ;

PRODUCTION: statement_list -> statement_list statement

TOKEN: READ

TOKEN: LPAREN

TOKEN: IDENTIFIER (a)

TOKEN: RPAREN

PRODUCTION: read_statement -> read (identifier)

PRODUCTION: simple_statement -> read_statement

TOKEN: SEMI

PRODUCTION: statement -> simple_statement ;

PRODUCTION: statement_list -> statement_list statement

TOKEN: READ

TOKEN: LPAREN

TOKEN: IDENTIFIER (b)

TOKEN: RPAREN

PRODUCTION: read_statement -> read (identifier)

PRODUCTION: simple_statement -> read_statement

TOKEN: SEMI

PRODUCTION: statement -> simple_statement ;

PRODUCTION: statement_list -> statement_list statement

TOKEN: READ

TOKEN: LPAREN

TOKEN: IDENTIFIER (c)

TOKEN: RPAREN

PRODUCTION: read_statement -> read (identifier)

PRODUCTION: simple_statement -> read_statement

TOKEN: SEMI

PRODUCTION: statement -> simple_statement ;

PRODUCTION: statement_list -> statement_list statement

TOKEN: IF

TOKEN: LPAREN

TOKEN: IDENTIFIER (a)

TOKEN: GE

relation -> >=

TOKEN: IDENTIFIER (b)

TOKEN: RPAREN

condition -> expression relation expression

TOKEN: LBRACE

TOKEN: IF

TOKEN: LPAREN

TOKEN: IDENTIFIER (a)

TOKEN: GE

relation -> >=

TOKEN: IDENTIFIER (c)

TOKEN: RPAREN

condition -> expression relation expression

TOKEN: LBRACE

TOKEN: IDENTIFIER (max)

TOKEN: ASSIGN

TOKEN: IDENTIFIER (a)

TOKEN: SEMI

PRODUCTION: assign_statement -> identifier = expression

PRODUCTION: simple_statement -> assign_statement

PRODUCTION: statement -> simple_statement ;

PRODUCTION: statement_list -> statement

TOKEN: RBRACE

if_statement -> if (condition) { statement_list }

TOKEN: ELSE

TOKEN: LBRACE

TOKEN: IDENTIFIER (max)

TOKEN: ASSIGN

TOKEN: IDENTIFIER (c)

TOKEN: SEMI

PRODUCTION: assign_statement -> identifier = expression

PRODUCTION: simple_statement -> assign_statement

PRODUCTION: statement -> simple_statement ;

PRODUCTION: statement_list -> statement

TOKEN: RBRACE

else_clause -> else { statement_list }

decision_statement -> if_statement else_clause

struct_statement -> decision_statement

PRODUCTION: statement -> struct_statement

PRODUCTION: statement_list -> statement

TOKEN: RBRACE

if_statement -> if (condition) { statement_list }

TOKEN: ELSE

TOKEN: LBRACE

TOKEN: IF

TOKEN: LPAREN

TOKEN: IDENTIFIER (b)

TOKEN: GE

relation -> >=

TOKEN: IDENTIFIER (c)

TOKEN: RPAREN

condition -> expression relation expression

TOKEN: LBRACE

TOKEN: IDENTIFIER (max)

TOKEN: ASSIGN

TOKEN: IDENTIFIER (b)

TOKEN: SEMI

PRODUCTION: assign_statement -> identifier = expression

PRODUCTION: simple_statement -> assign_statement

PRODUCTION: statement -> simple_statement ;

PRODUCTION: statement_list -> statement

TOKEN: RBACE

if_statement -> if (condition) { statement_list }

TOKEN: ELSE

TOKEN: LBACE

TOKEN: IDENTIFIER (max)

TOKEN: ASSIGN

TOKEN: IDENTIFIER (c)

TOKEN: SEMI

PRODUCTION: assign_statement -> identifier = expression

PRODUCTION: simple_statement -> assign_statement

PRODUCTION: statement -> simple_statement ;

PRODUCTION: statement_list -> statement

TOKEN: RBACE

else_clause -> else { statement_list }

decision_statement -> if_statement else_clause

struct_statement -> decision_statement

PRODUCTION: statement -> struct_statement

PRODUCTION: statement_list -> statement

TOKEN: RBACE

else_clause -> else { statement_list }

decision_statement -> if_statement else_clause

struct_statement -> decision_statement

PRODUCTION: statement -> struct_statement

PRODUCTION: statement_list -> statement_list statement

TOKEN: PRINT

TOKEN: LPAREN

TOKEN: IDENTIFIER (max)

TOKEN: RPAREN

PRODUCTION: print_statement -> print (print_type)

PRODUCTION: simple_statement -> print_statement

TOKEN: SEMI

PRODUCTION: statement -> simple_statement ;

PRODUCTION: statement_list -> statement_list statement

TOKEN: RBRACE

Parsing completed successfully.

p1err:

```
{  
# p1err.*  
    int x;  
    int 2var; # identifier cannot start with a digit  
}
```

OUTPUT:

TOKEN: LBRACE

TOKEN: INT

simple_type -> int

PRODUCTION: type -> simple_type

TOKEN: IDENTIFIER (x)

PRODUCTION: declaration_statement -> type identifier

PRODUCTION: simple_statement -> declaration_statement

TOKEN: SEMI

PRODUCTION: statement -> simple_statement ;

PRODUCTION: statement_list -> statement

TOKEN: INT

simple_type -> int

PRODUCTION: type -> simple_type

TOKEN: INT_CONST (2)

Syntax error on line 7: syntax error